

## Choice - 1

### CareLog – Secure Caregiver & Pet Tracker

#### Overview

CareLog is a daily tracking web app for families managing care for an elderly parent or pet, where all sensitive data — caregiver profiles and daily medical/care logs — is encrypted before storage and decrypted only on authorized retrieval. Daily care logs act as the "posts," and caregiver details act as the "profile" required by the lab. The goal is to build a working system where personal schedules and sensitive medical routines stay protected even if the database is leaked.

#### Users & Roles

- **Regular User (Caregiver):** register/login, add/view/edit/delete own daily care logs, update own profile.
- **Admin (Lead Caregiver):** manage account status and help with recovery, but cannot decrypt users' care log data.

#### Key Features

- **Registration & Login:** passwords salted + hashed from scratch; email/contact info encrypted with RSA.
- **Two-Factor Authentication:** password + a self-implemented HMAC-based OTP code.
- **Key Management Module:** generates, stores, and rotates each user's RSA and ECC keypairs.
- **Care Logs (Posts):** encrypted using ECC/ECIES; each record gets an HMAC/CBC-MAC tag to detect tampering.
- **Profile:** name/email/phone/emergency contact encrypted using RSA.
- **Sessions:** random unpredictable tokens, server-side checks, expiry, to prevent hijacking.

#### Cryptography Plan

Only asymmetric encryption is used — no symmetric ciphers. RSA handles profile data (small, infrequent), while ECC/ECIES handles care logs (frequent, higher volume, smaller ciphertext). All algorithms (RSA, ECC, hashing/HMAC, OTP) are implemented from scratch, without built-in crypto libraries or framework functions.

#### Tech Stack

Backend: Flask (Python) — Database: MongoDB (stores only ciphertext + MAC tags) — Frontend: HTML/CSS, vanilla JS.

---

## Choice - 2

### BookVault – Secure Privacy-First Library Management

#### Overview

BookVault is a library management web app focused on patron privacy, where all sensitive data — reading histories and user profiles — is encrypted before storage and decrypted only on authorized retrieval. Checkout records and book reviews act as the "posts," and patron details act as the "profile" required by the lab. The goal is to build a working system where a patron's reading habits stay completely protected from surveillance or database leaks.

## Users & Roles

- **Regular User (Patron):** register/login, add/view checkout records and reviews, update own profile.
- **Admin (Head Librarian):** manage book inventory and account status, but cannot decrypt users' reading history data.

## Key Features

- **Registration & Login:** passwords salted + hashed from scratch; email/contact info encrypted with RSA.
- **Two-Factor Authentication:** password + a self-implemented HMAC-based OTP code.
- **Key Management Module:** generates, stores, and rotates each user's RSA and ECC keypairs.
- **Checkout Records & Reviews (Posts):** encrypted using ECC/ECIES; each record gets an HMAC/CBC-MAC tag to detect tampering.
- **Profile:** name/email/phone/library card info encrypted using RSA.
- **Sessions:** random unpredictable tokens, server-side checks, expiry, to prevent hijacking.

## Cryptography Plan

Only asymmetric encryption is used — no symmetric ciphers. RSA handles profile data (small, infrequent), while ECC/ECIES handles checkout logs and reviews (frequent, higher volume, smaller ciphertext). All algorithms (RSA, ECC, hashing/HMAC, OTP) are implemented from scratch, without built-in crypto libraries or framework functions.

## Tech Stack

Backend: Flask (Python) — Database: Supabase/Postgres (stores only ciphertext + MAC tags) — Frontend: HTML/CSS, vanilla JS.